

Semana 4: Estruturas de Repetição (Loops) em JavaScript

O Motor da Automação no Client-Side



Objetivos de Aprendizagem da Semana



- **Automatizar Tarefas**

Substituir execuções manuais e redundantes por iterações de código.

- **Aplicar Laços Fundamentais**

Compreender e diferenciar as sintaxes estruturais de for, while e do...while.

- **Controlar o Fluxo Lógico**

Utilizar comandos avançados (break e continue) para manipular iterações.

- **Evitar Falhas Críticas**

Identificar e prevenir o loop infinito e o travamento do navegador.

O Paradigma da Repetição

Sem estruturas de repetição, o software é apenas um roteiro fixo e exaustivo. Na programação Web profissional, delegamos o trabalho repetitivo para a máquina processar volumes massivos de dados em milissegundos.

O Trabalho Braçal (Redundante)

```
console.log(1);  
console.log(2);  
console.log(3);  
console.log(4);  
...  
console.log(10);
```

A Máquina de Automação (Eficiente)

```
for (let i = 1; i <= 10; i++) {  
  console.log(i);  
}
```


A Estrutura for: O Laço Determinado

```
for (let i = 0; i < 10; i++) { ... }
```

Condição

Até onde vai. O loop roda enquanto isso for true.

Incremento

O passo da engrenagem. Atualiza o contador após cada volta.

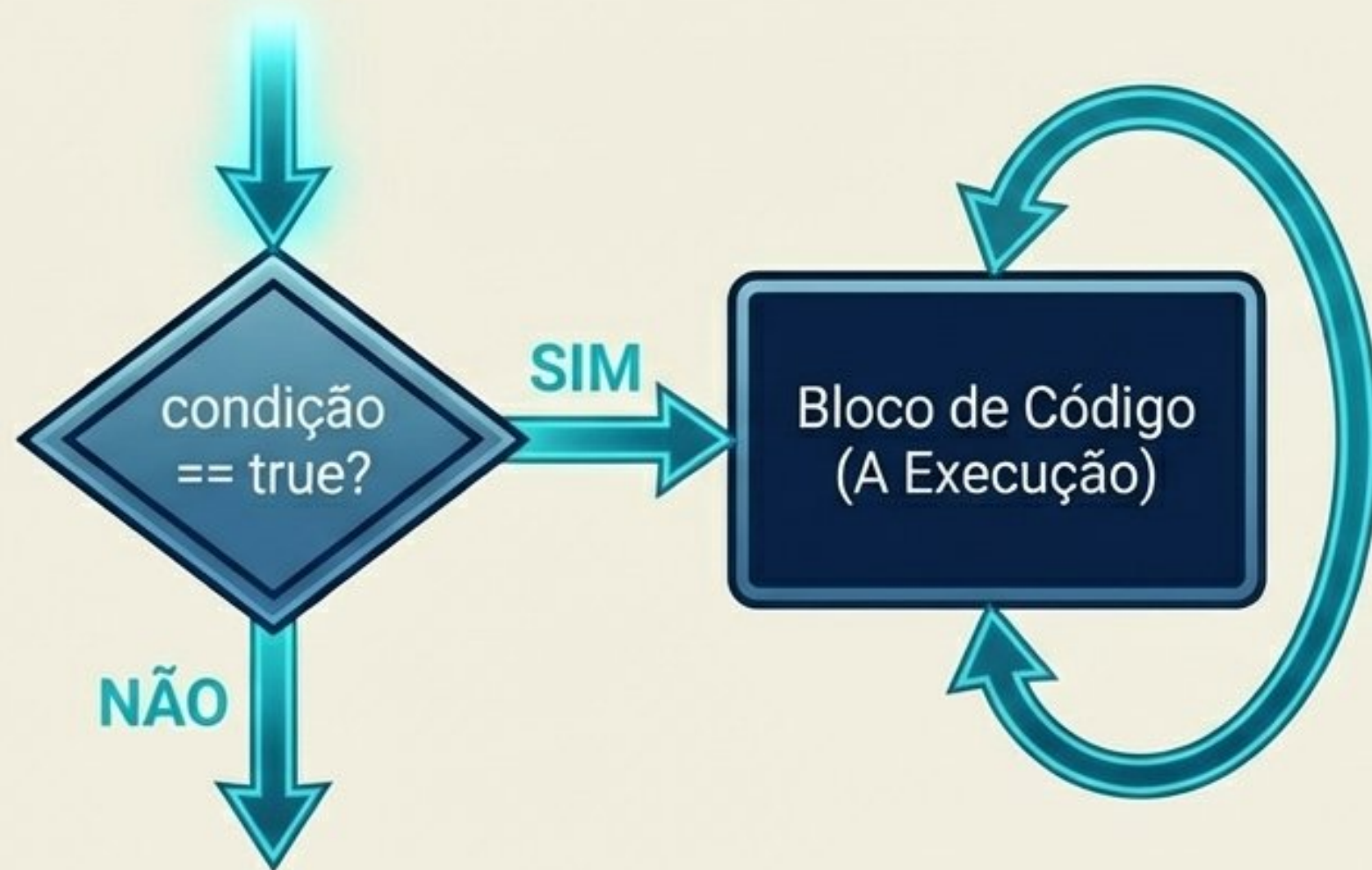
Inicialização

Onde o motor liga. Executado apenas uma vez.



Aplicação: Utilize o for quando o número exato de repetições for conhecido previamente (ex: iterar de 1 a 100).

A Estrutura while: O Laço Condicional



```
while (condicao) {  
    // Executa enquanto for verdadeiro  
}
```

- A Catraca de Entrada: A condição é testada ANTES de executar o bloco. Se for falsa logo de cara, o código interno nunca será lido.
- Aplicação: Ideal para cenários incertos, como aguardar uma ação específica do usuário ou ler dados até encontrar uma chave de parada.

A Estrutura do...while: A Execução Garantida



```
do {  
    // Executa pelo menos uma vez  
} while (condicao);
```

- A Catraca de Saída: A validação foi movida para o final. O bloco de código é executado obrigatoriamente pelo menos uma vez, independentemente da condição.
- Aplicação: Perfeito para menus interativos, onde o usuário precisa ver as opções na tela antes de decidir se deseja sair.

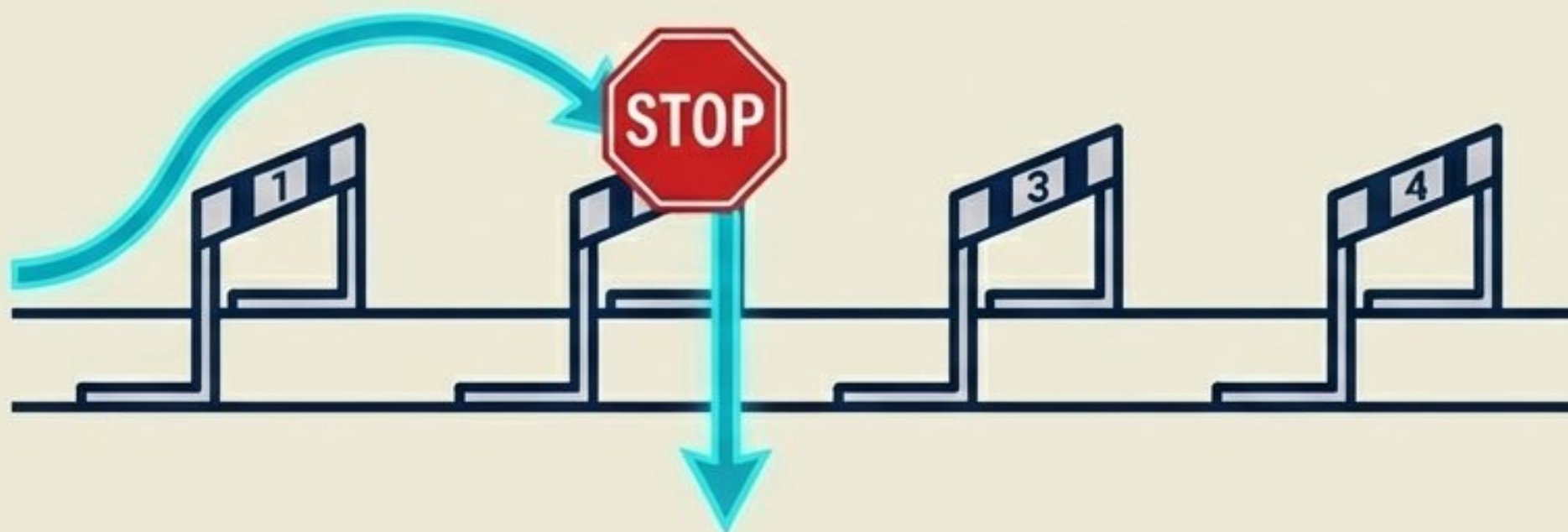
Matriz de Decisão: Qual laço escolher?

	for	while	do...while
Número de iterações conhecido?	Sim	Não (depende de evento)	Não
Momento da validação?	Antes da execução	Antes da execução	Após a execução
Garantia de execução?	Nenhuma (0 a N vezes)	Nenhuma (0 a N vezes)	Mínimo de 1 vez (1 a N vezes)

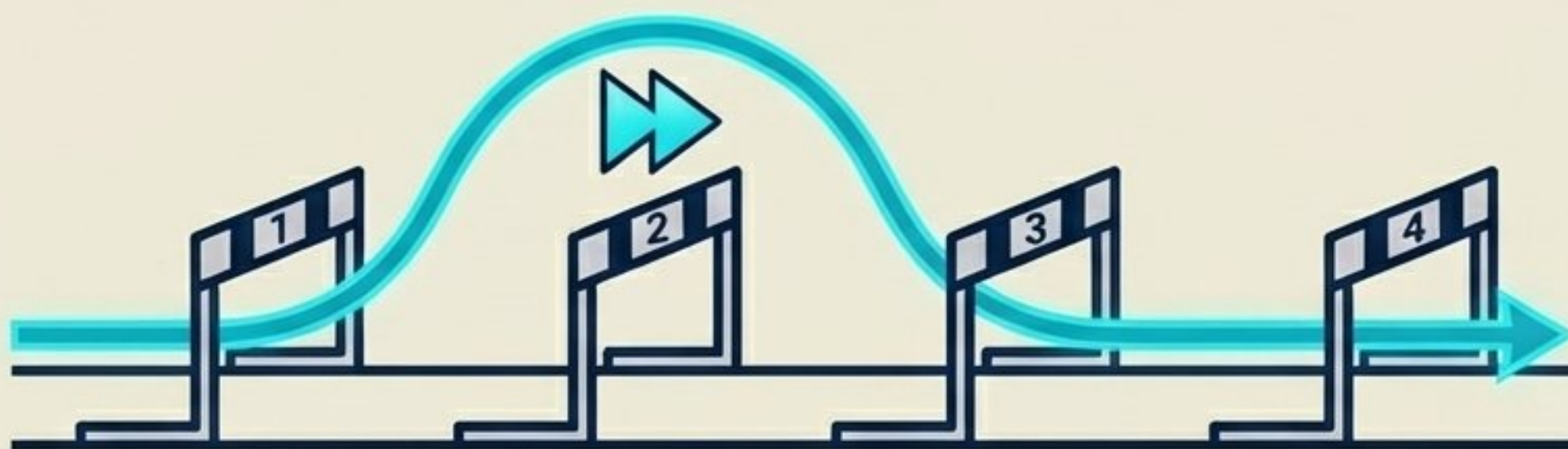


Dica de Ouro: Na dúvida, o while é a estrutura condicional mais versátil e segura para fluxos dinâmicos no desenvolvimento Web.

Controle de Fluxo Fino: **break** e **continue**



break: O Botão de Emergência. Interrompe o laço imediatamente e sai da estrutura, ignorando qualquer iteração restante. Útil ao encontrar o dado exato em uma busca.



continue: O Botão de Pulo. Ignora o restante do código apenas daquela iteração específica e avança direto para o próximo ciclo. Útil para filtrar valores indesejados.

O Perigo Crítico: O Loop Infinito

Um laço sem uma rota de fuga travada consumirá toda a memória do navegador em segundos, travando a aplicação.

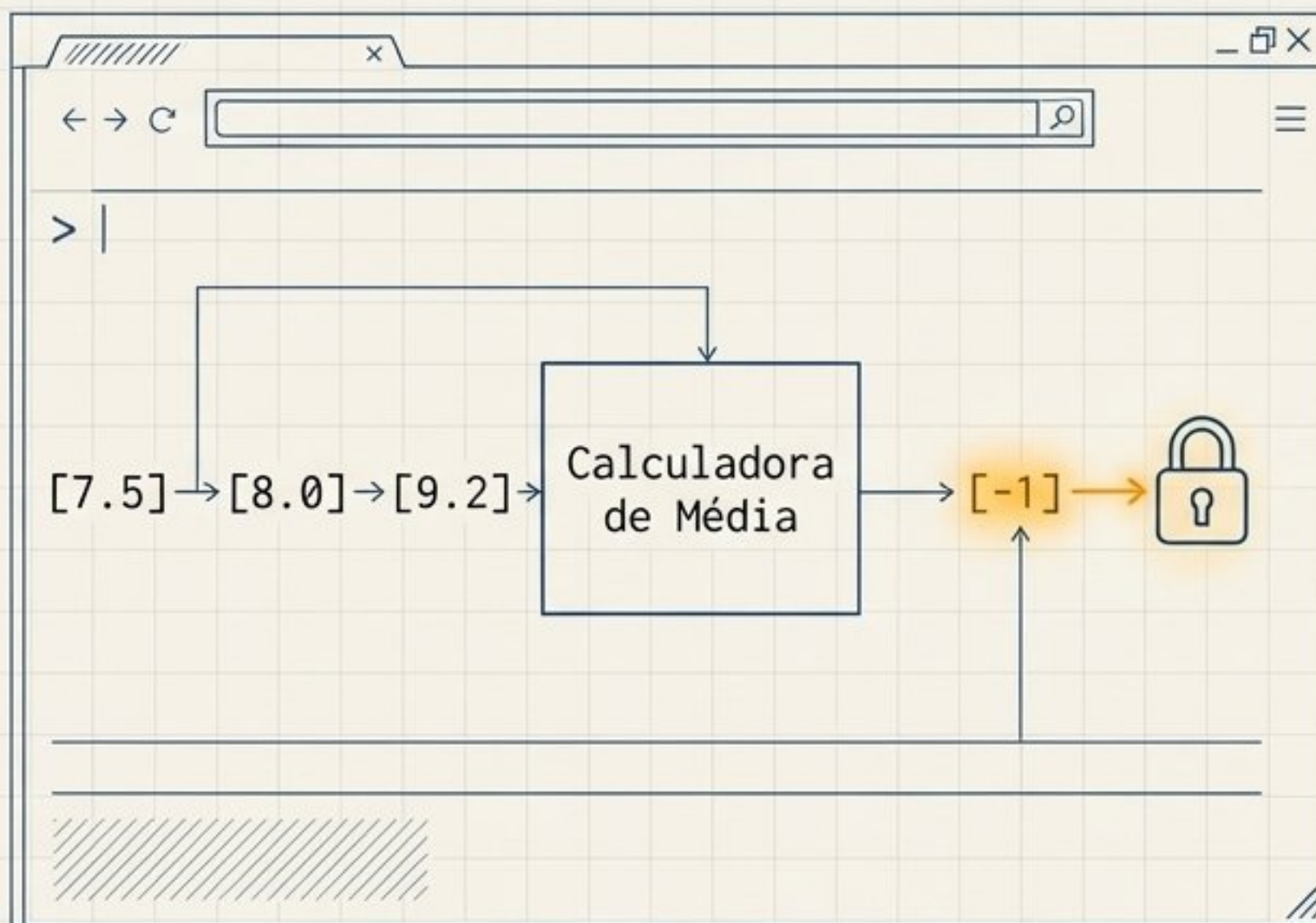
Causas Comuns:

- ⚠ - Esquecer de atualizar a variável de controle (ex: omitir o `i++` dentro de um bloco `while`).
- ⚠ - Criar uma condição que matematicamente nunca se tornará falsa (ex: `while (i > 0)` em um cenário onde a variável `i` apenas aumenta).



A Regra de Ouro: Antes de escrever o que o laço faz, escreva a sua Condição de Parada. Todo labirinto precisa de uma saída.

Situação-Problema (ABP): 0 Sistema de Notas Dinâmico



Entrada de Dados e Sentinela (-1)

0 Cenário:

Você precisa desenvolver um script para um professor calcular a média da turma. O problema? A quantidade de alunos varia todos os dias.

0 Desafio Lógico:

- ☒ - Solicitar notas sequencialmente utilizando o comando prompt.
- ☒ - Acumular a soma total das notas e contar internamente quantos alunos foram registrados.
- ☒ - A Chave de Parada: O script só deve interromper a repetição quando o professor digitar o valor sentinela -1.
- ☒ - Ao final do processo, exibir a soma total e a média matemática da turma no console.

Roteiro da Prática (Laboratório)



Etapa 1: O Aquecimento (Contadores)

Criar um contador progressivo (1 a 10) e regressivo (10 a 1) no console para validar o domínio sintático do for e do while.



Etapa 2: A Tabuada Dinâmica

Trabalho em duplas. Desenvolver um gerador onde o usuário escolhe o multiplicador. O laço iterará de 1 a 10, construindo a saída visual da tabuada.



Etapa 3: O Filtro Matemático

Criar um laço que percorra de 1 a 50, utilizando o operador de módulo (%) combinado com o comando continue para imprimir apenas os números pares.

Atividade Verificadora e Critérios de Sucesso

A Entrega (Individual)

Um algoritmo funcional que processe uma série de valores numéricos de forma autônoma, apresentando uma condição de parada clara e executando uma operação matemática contínua.

Critérios Formativos (Como você será avaliado):

1. Escolha Estrutural

O laço selecionado (for, while ou do...while) faz sentido para o fluxo de dados proposto na solução?

2. Segurança da Lógica

A condição de parada foi implementada corretamente?
O risco de loop infinito foi completamente mitigado?

3. Clean Code

As variáveis utilizam let/const adequadamente? O código possui indentação clara e nomenclatura semântica compreensível?

Próximos Passos: O Limite das Variáveis Simples

O Motor de Automação está pronto. Você já sabe como repetir ações infinitamente. Mas existe um gargalo logístico: Se lermos 100 notas com um loop while, como guardar todas elas na memória ao mesmo tempo sem criar 100 variáveis diferentes?

The Bottleneck



The Solution preview



Semana 5: Vetores (Arrays). Como armazenar, estruturar e iterar sobre coleções complexas de dados.